第六讲作业. 马福泉 23336179

1、a→b，事件b的发生不一定与a有关系。(在现实中)

在分布式系统中，a→b是一种逻辑上的因果顺序，用于通过时间戳，消息传递等机制来判断事件的先后与依赖关系。这是分布系统中为了维护一致性和顺序性而引入的逻辑概念，并不等同于现实世界中的因果关系。

2、(1)场景定义：N_1 N_2两个节点，协同编辑文档D，支持并发编辑

需解决：①操作因果顺序追踪 ②并发冲突检测

(2)操作算法过程：①初始化：每一用户节点维护一个向量时钟，长度=节点数，(初始为0

②操作与传播

N_1编辑：用户输入"Hello"，记为op1

N_1递增自身计数器，$VC_1=[N_1:1, N_2:0]$

并把op1与VC_1绑定，广播为$e_1=(op_1, VC_1)$

同理：N_2编辑：用户删除该行，记为op2

N_2递增VC_2的N_2 $VC_2=[N_1:0, N_2:1]$

op2与VC_2绑定，广播为$e_2=(op_2, VC_2)$

③接收处理与冲突检测：

如：N_2收到e_1对比VC_1与VC_2

两者互不包含，说明op1与op2并发且产生了冲突

提示用户手动选择编辑解决冲突

(3)向量时钟的作用：

①因果追踪保障：可以记录操作OP

②可以检测冲突：当2个VC互不包含，说明并发冲突

③可以保持一致性：合并VC可实现文档协同一致编辑。

3. (1) 第二次互斥算法为：基于 Lamport 逻辑时钟的全序多播。

它与 Ricart & Agrawala 分布式算法最相近。

(2) 相同点：① 维护请求队列，时间顺序处理请求

② 分布式协作：无需中央协调器，进程间通过消息交换协作实现互斥访问权。

(3) 不同点：① 前者通过队列和超时机制可实现高容错性，后者对单进程故障敏感，易阻塞，容错率低。

② 请求机制不同：前者通过全序多播广播请求，需要等待所有进程的 ALLOW 消息，依赖逻辑时钟实现全序。后者直接向其他所有进程发送请求，接收同意信息后进入临界区

③ 复杂度：前者需维护多播队列

后者算法简单，但要处理更多消息。

4. (1) 消息唯一性：每个 Election 消息携带发起点 ID（全局唯一）以及序列号（发起点本地递增，全局唯一），从而标识消息来源与版本。

(2) 相遇时比较优先级：

当节点收同时收到两个 ID 发来的 Election 消息：保留 ID 更大的消息，丢弃低 ID 消息

(3) 状态管理

节点维护一个已处理消息表（记录 ID、序列号）

避免重复处理，

丢弃低 ID 消息：可直接标记为“已处理”，加入到上述已处理消息表。